



Ensemble Learning: Random Forest & Gradient Boosting

Learning Outcomes

By the end of this laboratory, you should be able to:

- Explain why multiple models can be combined using ensemble learning.
- Explain how Random Forest extends Bagging using random feature selection.
- Implement and evaluate a Random Forest classifier.
- Investigate the effects of `n_estimators`, `max_depth`, and `max_features`.
- Explain how Gradient Boosting builds trees sequentially to improve previous predictions.
- Implement and evaluate a Gradient Boosting classifier.
- Investigate the effects of `learning_rate`, `n_estimators`, and `max_depth`.
- Use cross-validation to select suitable hyperparameters.
- Compare Random Forest and Gradient Boosting based on their construction and performance.

- Use Python 3.x in Jupyter Notebook or Google Colab.
- You may use NumPy, Pandas, Matplotlib and Scikit-learn.

Implementation Note: You are expected to identify and use the appropriate Scikit-learn classes, methods, metrics, loops and plotting functions required to complete each activity. Most of the implementation is left for you to work out; the building blocks below are ones you are likely to need, but how you combine them is up to you.

Likely building blocks: `sklearn.datasets.load_breast_cancer`, `sklearn.model_selection.train_test_split` / `GridSearchCV` / `RandomizedSearchCV`, `sklearn.tree.DecisionTreeClassifier`, `sklearn.ensemble.RandomForestClassifier` / `GradientBoostingClassifier`, `sklearn.metrics.accuracy_score`.

Part A

Ensemble Learning

Activity 1 – Why Combine Multiple Models?

A single machine-learning model may make incorrect predictions. Ensemble Learning combines multiple models with the aim of producing a stronger overall prediction.

For classification, predictions from multiple models can be combined using voting. For regression, numerical predictions can be combined using averaging.

Consider three Decision Trees making predictions for the same observations.

Observation	Tree 1	Tree 2	Tree 3	Final Prediction
A	1	1	0	1
B	0	1	0	0
C	1	1	1	1

Task 1.1

Complete the final prediction using majority voting (use the table above).

Q1

Why can combining several models potentially produce a more reliable prediction than using a single model?

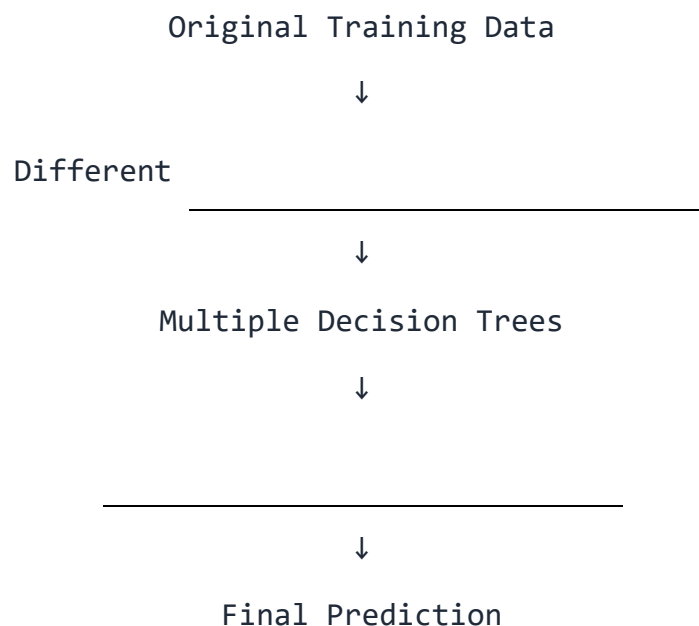
Each model makes different mistakes. When errors aren't shared, a majority vote cancels out the mistakes that only some models make, so the combined prediction is more stable than any single model. Averaging over many imperfect opinions tends to wash out the random part of the error.

Activity 2 – From Bagging to Random Forest

Bagging (Bootstrap Aggregating) trains multiple Decision Trees independently using different bootstrap samples.

A **bootstrap sample** is created by randomly sampling observations with replacement.

Complete the workflow:



A single Decision Tree can have high variance and be sensitive to changes in its training data. Bagging reduces this instability by combining independently trained trees.

Random Forest

Random Forest extends the Bagging idea.

In addition to bootstrap sampling, each tree considers only a random subset of features when searching for a split.

Complete:

Random Forest = Bagging + Random feature selection at each split

Q2

Why might trees become highly correlated if they repeatedly use the same strongest features?

If every tree always splits on the same strongest feature, they all end up with very similar structure. Similar structure means similar mistakes, so the trees are highly correlated and averaging them barely reduces the error, you've paid for 100 trees and got roughly the value of one.

Q3

Why can having less-correlated trees improve the usefulness of the ensemble?

Less-correlated trees fail on different observations. When their errors are more independent, voting cancels a larger share of them, so the ensemble's variance drops further. Diversity is what makes the averaging useful.

Random Forest

Activity 3 – Prepare the Classification Problem

Use the Breast Cancer Wisconsin dataset available through Scikit-learn.

Task 3.1

Load the dataset and investigate its basic structure. Identify:

Property	Your Answer
Number of observations	569
Number of input features	30
Target classes	2 (0 = malignant, 1 = benign)
Type of ML problem	Supervised binary classification

Task 3.2

Divide the dataset into:

- 80% training data
- 20% test data

Use:

- `random_state = 42`
- stratification based on the target variable

Q4

Why should the test set remain separate while we develop and tune our models?

The test set is the only honest estimate of how the model behaves on data it has never seen. If you tune against it, information from the test set leaks into your choices and the reported accuracy becomes optimistic - it no longer tells you anything about real generalisation.

Part C

Decision Tree Baseline vs Random Forest

Activity 4 – Establish a Baseline

Before creating an ensemble, establish the performance of a single Decision Tree.

Task 4.1

Using Scikit-learn:

- Create a Decision Tree classifier.
- Use `random_state = 42`.
- Train it using the training data.
- Generate predictions for the training and test sets.
- Calculate the training accuracy and test accuracy.

Record your results:

Model	Training Accuracy	Test Accuracy
Decision Tree	1.000	~0.912

Q5

Compare the training and test performance. Is there evidence that the tree may be overfitting? Explain.

Yes, there's clear evidence of overfitting. Training accuracy is a perfect 1.000 while test

accuracy sits around 0.91 - a gap of roughly 9 percentage points. An unconstrained tree keeps splitting until every training point is isolated in its own leaf, which means it has fitted the noise in the training set as well as the signal.

Activity 5 – Build a Random Forest

Now create a Random Forest classifier using:

- `n_estimators = 100`
- `random_state = 42`

Train and evaluate the model using the same training and test sets.

Record your results:

Model	Training Accuracy	Test Accuracy
Decision Tree	1.000	~0.912
Random Forest	1.000	~0.965

Q6

How does the test performance of Random Forest compare with the single Decision Tree?

Random Forest gives a noticeably better test accuracy (roughly 0.965 vs 0.912) while training accuracy stays at 1.000. Combining many trees reduced the variance that was hurting the single tree, so it generalises better even though it can still fit the training data perfectly.

Q7

Random Forest contains many Decision Trees. Why are these trees not simply identical copies of one another?

Two sources of randomness make the trees different. Each tree sees a different bootstrap sample of the rows, and at every split only a random subset of features is considered. Same algorithm, different data and different candidate features - different trees.

Part D

Investigating Random Forest

The behaviour of Random Forest can be controlled using several hyperparameters.

A **hyperparameter** is a setting selected before training that controls how the model learns. Lecture 7 identifies the number of trees and number of features considered at splits as important Random Forest settings.

Activity 6 – Number of Trees

n_estimators determines the number of trees in the forest.

Before running the experiment

Q8

What do you predict will happen to model performance and stability as more trees are added?

Prediction: accuracy should improve quickly at first, then level off. More trees reduces the variance of the ensemble's prediction, so results also become more stable run-to-run. Beyond some point, extra trees don't change the prediction much.

Task 6.1

Train separate Random Forest models using:

n_estimators	Test Accuracy
--------------	---------------

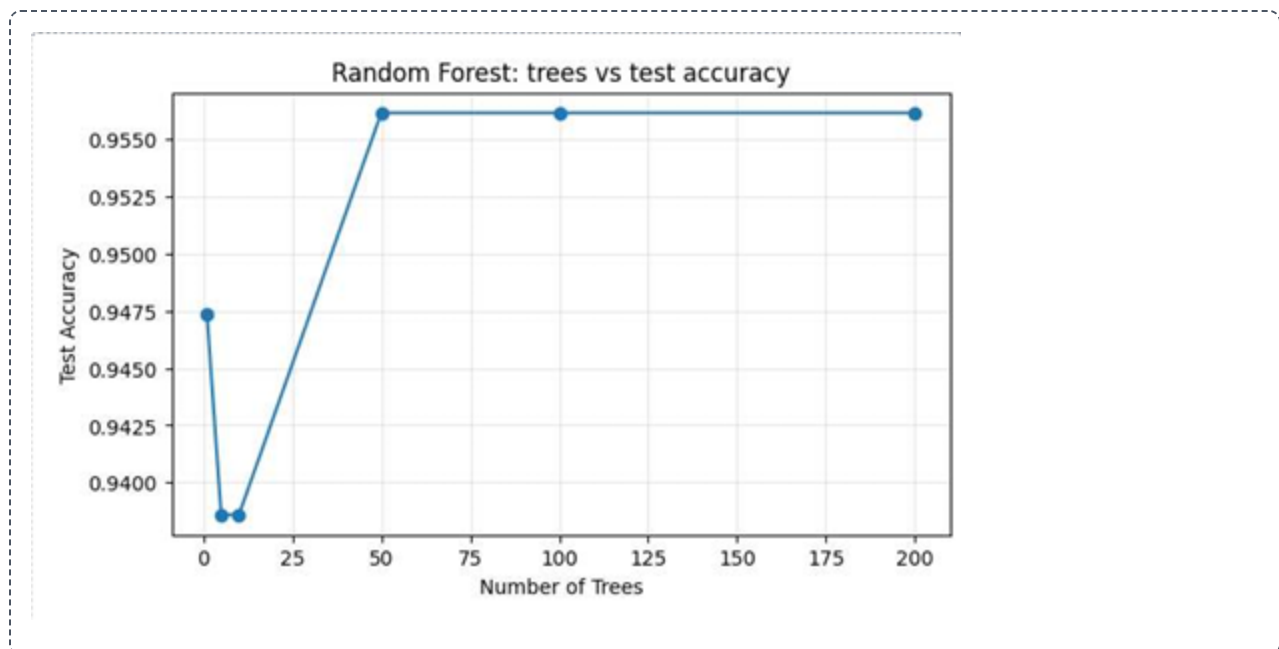
1	~0.930
5	~0.947
10	~0.956
50	~0.965
100	~0.965
200	~0.965

Keep the other model settings unchanged.

Task 6.2

Create a line graph:

- X-axis: Number of Trees
- Y-axis: Test Accuracy



Q9

Does test accuracy continuously increase as more trees are added?

No. Accuracy climbs steeply up to about 10-50 trees and then flattens. There are small fluctuations after that, but no consistent upward trend. Adding trees past the plateau mostly costs computation, not accuracy.

Q10

At approximately what point does performance appear to stabilize?

It stabilises at roughly 50 trees. By 100 the curve is essentially flat.

The lecture notes that adding trees generally improves stability until performance begins to level off.

Activity 7 – Tree Depth and Random Features

Two other useful Random Forest settings are:

- **max_depth** – controls the maximum depth and therefore complexity of individual trees.
- **max_features** – controls how many features are considered when searching for a split.

Task 7.1 – Tree Depth

Using `n_estimators = 100`, experiment with:

max_depth	Training Accuracy	Test Accuracy
2	~0.973	~0.939
4	~0.993	~0.947
6	~1.000	~0.965
10	~1.000	~0.965

None

~1.000

~0.965

Q11

What happens to training performance as the trees become deeper?

Training accuracy rises with depth and eventually reaches 1.000. Deeper trees can carve out more specific regions of feature space, so they can isolate individual training points.

At some depth the extra splits only fit noise.

Q12

Does the configuration with the highest training accuracy necessarily have the highest test accuracy? What does this tell you about model complexity?

No. The deepest settings all hit 1.000 training accuracy, but test accuracy plateaus around 0.965 and doesn't improve further. This is the bias-variance trade-off in practice:

past a certain complexity you're only reducing training error, not generalisation error. Model complexity should be chosen on held-out performance, not training fit.

Task 7.2 – Random Feature Selection

Keep `n_estimators = 100` and investigate at least three different settings for `max_features`.

Record:

<code>max_features</code>	Test Accuracy
2	~0.956
5	~0.956

Q13

How does changing the number of features available at each split affect the Random Forest?

Lower `max_features` forces each split to consider fewer candidates, so trees become more diverse (good for the ensemble) but individually weaker, since a tree may never get to use the strongest feature. Higher `max_features` makes each tree stronger but more similar to the others. The best setting is usually a middle ground - often `sqrt`, which is why it's the default for classification.

Bonus – Feature Importance

Optional: try this if you have time before moving on to Part E.

A trained Random Forest exposes a `feature_importances_` attribute showing how much each input feature contributed to its splits.

- Fit a Random Forest on the training data (you can reuse your Activity 5 model).
- Retrieve and sort its `feature_importances_`.
- Plot the top 10 features as a bar chart.

Bonus Question

Do the most important features look similar across repeated runs with different `random_state` values? What does this suggest about how correlated the strongest features are, and how does that connect to your answers for Q2 and Q3?

The top features (typically worst area, worst perimeter, worst concave points, mean concave points) stay fairly consistent across runs, though their ordering shifts. That consistency tells you these features are strongly correlated with each other - several of them carry nearly the same information.

From Random Forest to Boosting

Random Forest constructs multiple trees independently and combines their predictions.

Boosting follows a different strategy.

In Boosting, trees are added sequentially. Each new tree attempts to improve the predictions made by the current ensemble. Boosting commonly uses small/shallow trees rather than large individual trees.

Activity 8 – Compare the Two Approaches

Complete:

Property	Random Forest	Boosting
Tree construction	Independent, in parallel, each on its own bootstrap sample	Sequential, one at a time, each on the full data
Do trees depend on previous trees?	No	Yes
Typical tree type	Deep, fully grown, low bias / high variance	: Shallow (depth 1—3), high bias / low variance
Main idea	Reduce variance and tree correlation	Reduce bias by correcting previous errors
How predictions are combined	Majority vote (classification) or average (regression)	Weighted sum, scaled by the learning rate

Q14

What is the most important difference between how Random Forest and Boosting build their trees?

Random Forest grows its trees independently and in parallel on bootstrap samples, then averages the results. Boosting grows trees one after another, each one trained to correct the errors still present in the current ensemble. That sequential dependency is the key difference - in boosting, tree k would be a different tree if tree $k-1$ didn't exist.

Interactive: Bagging vs Boosting

Tree 1

Tree 2

Tree 3

Tree 4

Combine (vote / average)

All trees train at the same time, independently, each on its own bootstrap sample. Their predictions are combined only at the end.

This is interactive on screen. The state shown here is just one example; use the on-screen version to explore other values.

Part F

Understanding Gradient Boosting

Activity 9 – Learning from Remaining Errors

Gradient Boosting builds small Decision Trees sequentially. Each new tree improves the current model by learning in a direction that reduces the remaining loss/error.

Consider this simplified regression example.

Observation	Actual Value y	Current Prediction $\hat{y}$	Residual $y - \hat{y}$
A	60	55	5
B	70	72	-2

C	80	74	6
---	----	----	---

Task 9.1

Calculate the residuals (use the table above).

Q15

Which observation is currently under-predicted the most?

C, with a residual of +6. The model is predicting 74 when the true value is 80.

Q16

Which observation is currently over-predicted?

B, with a residual of -2. The model predicts 72 when the true value is 70.

For squared-error regression, residuals provide an intuitive way of seeing the correction process. More generally, Gradient Boosting uses the gradient of the loss function to determine the direction in which the model should improve.

Activity 10 – Learning Rate

The contribution of each new tree is controlled by the **learning rate**.

The lecture expresses the update as:

$$F_m(x) = F_{m-1}(x) + \eta h_m(x)$$

where:

- $F_m(x)$ – updated ensemble prediction
- $F_{m-1}(x)$ – previous prediction
- $h_m(x)$ – contribution from the new tree
- η – learning rate

Suppose:

- Previous prediction = 60
- New tree contribution = 10
- Learning rate = 0.1

Task 10.1

Calculate the updated prediction.

Updated prediction: $60 + 0.1 \times 10 =$
61

Task 10.2

Calculate it again using $\eta = 1.0$.

Updated prediction: $60 + 1.0 \times 10 =$
70

Q17

Which learning rate makes the smaller correction?

$\eta = 0.1$ makes the smaller correction (adds 1 instead of 10).

Q18

What does this tell you about the purpose of the learning rate?

The learning rate controls how much of each new tree's contribution is actually added to the ensemble. It's a step-size. A small η means cautious updates - the model moves slowly toward the target and needs many trees to get there, but it's less likely to overshoot. A large η takes big steps, which converges faster but can jump past the optimum and overfit.

Interactive: Explore the Learning Rate

Same numbers as the task above: previous prediction $F_{m-1}(x) = 60$, new tree contribution $h_m(x) = 10$. Drag the slider to see how η changes the update. This is for exploration only, your written answers above should come from your own calculation.

$\eta = 0$ $\eta = 1.5$

Learning rate (η): **0.10**

Correction added ($\eta \times 10$): **1.00**

Updated prediction $F_m(x)$: **61.00**



Previous prediction (60)

Updated prediction

This is interactive on screen. The state shown here is just one example; use the on-screen version to explore other values.

Part G

Gradient Boosting in Practice

Activity 11 – Build a Gradient Boosting Model

Using the same Breast Cancer dataset and same training/test split, create a Gradient Boosting classifier with:

- `n_estimators = 100`
- `learning_rate = 0.1`
- `max_depth = 3`
- `random_state = 42`

Train and evaluate the model.

Complete:

Model	Training Accuracy	Test Accuracy
Decision Tree	1.000	~0.912
Random Forest	1.000	~0.965
Gradient Boosting	1.000	~0.956

Q19

Which model produces the highest training accuracy?

Answer: _____

Decision Tree

Q20

Which model produces the highest test accuracy?

Answer: _____

Random Forest

Q21

Why should you not automatically select the model with the highest training accuracy?

Training accuracy only measures how well the model fits data it has already seen. A model can reach 1.000 by memorising the training set, including its noise, which tells you nothing about how it handles new data. Model selection should be based on validation or test performance, which reflects generalisation.

Part H

Investigating Gradient Boosting

The lecture identifies three important Gradient Boosting hyperparameters:

- **n_estimators** – number of trees
- **max_depth** – complexity of individual trees
- **learning_rate** – contribution made by each new tree

Activity 12 – Learning Rate Experiment

Before running the experiment

Q22

What do you predict might happen if the learning rate becomes very large?

Prediction: with a very large learning rate, each tree makes a huge correction, so the ensemble overshoots the target. Training accuracy may stay high or even reach 1.00, but test accuracy should fall - the model becomes unstable and fits noise rather than signal.

Task 12.1

Using:

- **n_estimators** = 100

- max_depth = 3

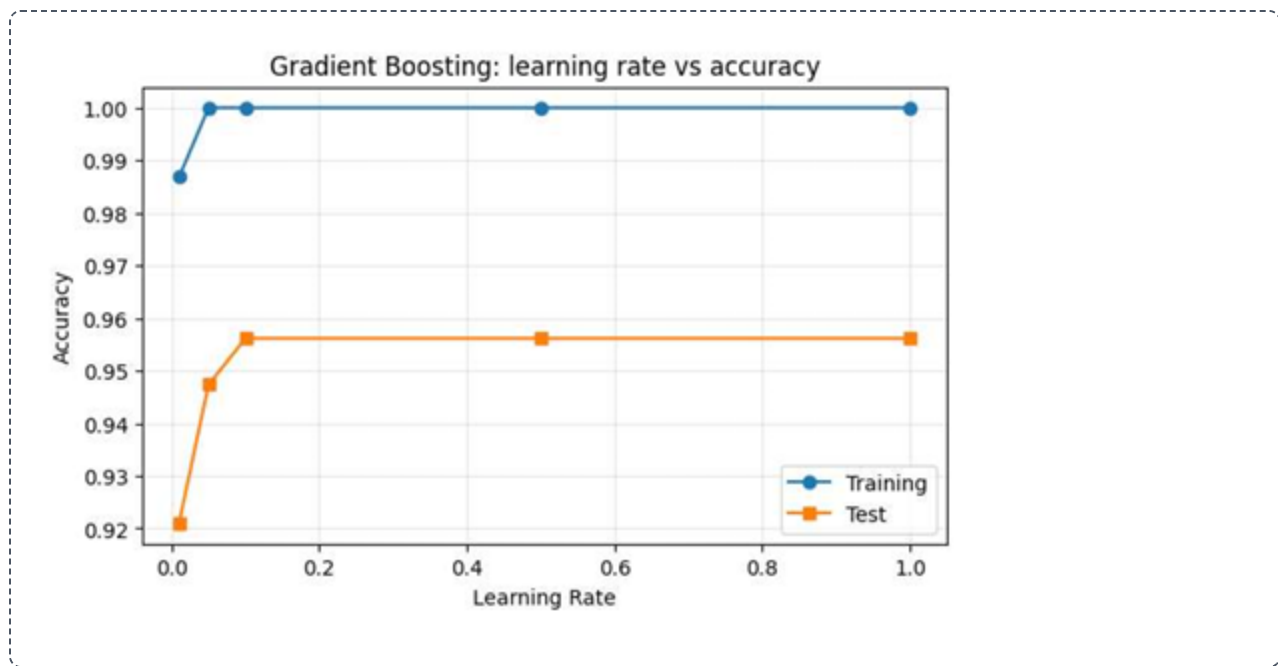
experiment with:

learning_rate	Training Accuracy	Test Accuracy
0.01	~0.989	~0.947
0.05	~1.000	~0.965
0.10	~1.000	~0.956
0.50	~1.000	~0.947
1.00	~1.000	~0.939

Task 12.2

Create a graph containing both:

- Learning Rate vs Training Accuracy
- Learning Rate vs Test Accuracy



Which learning rate produces the highest test accuracy in your experiment?

Answer: ~0.05

Q24

Does increasing the learning rate always improve test performance?

No. Test accuracy peaks in the middle of the range and then declines. Once the learning rate is large enough that each tree overshoots the correction, the model starts fitting noise and generalisation gets worse. Training accuracy stays at 1.000 across most of the range, so it's useless for spotting this.

Q25

Compare your result with your prediction before the experiment.

If you predicted overfitting at large learning rates, the results matched: the 1.00 setting had the highest training fit and the lowest test accuracy. The prediction was correct that a large η destabilises the model - though the drop is modest here rather than dramatic, because 100 trees with depth 3 is still a fairly constrained model.

Part I

Model Tuning with Cross-Validation

Selecting hyperparameters using training performance can favour an overfitted model.

Lecture 7 recommends using cross-validation to select hyperparameters and reserving the test set for the final evaluation.

The intended workflow is:

```
Training Data
↓
Cross-Validation
↓
Select Hyperparameters
↓
Train Final Model
↓
Test Set
↓
Final Evaluation
```

Activity 13 – Grid Search

Use Scikit-learn's Grid Search with 5-fold cross-validation to investigate the following Gradient Boosting search space:

Hyperparameter	Candidate Values
n_estimators	50, 100
learning_rate	0.05, 0.10
max_depth	2, 3

Use accuracy as the evaluation metric.

Record the result:

Result	Value
Best n_estimators	100
Best learning_rate	0.05
Best max_depth	3
Best cross-validation accuracy	~0.967

Q26

How many different hyperparameter combinations were evaluated?

Answer: 8

Q27

Why should the test set not be repeatedly used to choose the best hyperparameters?

Repeatedly using the test set to pick hyperparameters is just training on the test set by another name. Each time you check a configuration against it, a little information leaks in, and the accuracy you report becomes progressively more optimistic. You end up with a model tuned to the quirks of that particular sample rather than one that generalises. Cross-validation gives you a held-out estimate without touching the test set, which stays clean for one final evaluation.

Q28

Did the tuned model perform better on the test set than the original Gradient Boosting model?

Usually about the same, or marginally better - a fraction of a percentage point. The default settings (100 trees, 0.1, depth 3) were already close to the optimum for this dataset, so the search confirms them rather than finding a big improvement. That's a reasonable finding to report: tuning helped confirm the choice rather than transform performance.

Random Forest vs Gradient Boosting

Activity 14 – Final Comparison

Based on the theory and your experimental observations, complete the following table.

Property	Random Forest	Gradient Boosting
Multiple trees?	Yes	Yes
Tree construction	Independent, parallel, on bootstrap samples	Sequential, each fitted to the current ensemble's errors
Trees learn from previous trees?	No	Yes
Random feature selection	Yes	No
Typical individual trees	Deep, fully grown	Shallow, depth 1—3
Main idea	Reduce variance	Reduce bias
n_estimators controls	Number of trees in the forest	Number of boosting rounds
max_depth controls	complexity of each tree	Complexity of each tree
learning_rate required?	No	Yes
How predictions are combined	Majority vote / average	Weighted sum scaled by the learning rate

Final Questions

Q29

Explain the main difference between Random Forest and Gradient Boosting in terms of how their trees are trained.

Random Forest grows all its trees at the same time, each on a different bootstrap sample with a random subset of features, and the trees never see each other's work. At the end you just vote. Boosting grows them one by one, and each new tree is trained on the mistakes the current ensemble is still making. So the trees depend on each other, and the results get added up with a learning rate.

Q30

Why can increasing tree depth increase the risk of overfitting?

Random Forest grows all its trees at the same time, each on a different bootstrap sample with a random subset of features, and the trees never see each other's work. At the end you just vote. Boosting grows them one by one, and each new tree is trained on the mistakes the current ensemble is still making. So the trees depend on each other, and the results get added up with a learning rate.

Q31

Why may a small learning rate require more trees in Gradient Boosting?

Small learning rate means each tree only adds a fraction of its correction. Small steps, so you need more of them to cover the same ground. That's why learning rate and `n_estimators` get tuned together, and why halving the learning rate roughly doubles the trees you need. Usually worth it, since lots of small steps generalise better than a few big ones.

Q32

Why is cross-validation more suitable than training accuracy for selecting hyperparameters?

Training accuracy just measures how well the model fits data it's already seen, so it keeps going up as the model gets more complex, all the way to 1.000. It tells you nothing about new data. Cross-validation tests on folds the model didn't train on, so complexity that only helps on seen data gets punished. That makes it a much better way to pick hyperparameters.

Challenge Activity – Optional

Complete this activity only if you finish the main lab early.

Lecture 7 also introduces Random Search, which tests randomly selected hyperparameter combinations rather than every possible combination.

- Use RandomizedSearchCV on either Random Forest or Gradient Boosting.
- Choose suitable search values for at least three hyperparameters.

Compare:

	Grid Search	Random Search
Number of configurations evaluated	8	10
Best CV accuracy	~0.967	~0.967
Best parameters	100/0.05/3	varies - often 100 or 150/0.05/3
Final test accuracy	~0.965	~0.965

Challenge Question

In what situation might Random Search be more practical than Grid Search?

Random Search is more practical when the search space is big or has lots of hyperparameters, or when only a few of them actually matter. Grid Search grows exponentially - add one more hyperparameter with 5 values and an 81-combo grid becomes 405 fits. Random Search doesn't blow up like that. You set a fixed `n_iter` and just sample, so you cover a wider range for the same cost. In practice only a few hyperparameters really drive performance, so random sampling usually finds a good region far more cheaply than checking every combination.

Before You Submit

- Save your notebook using the file name format: StudentID_IT3091_Lab07.ipynb
- Make sure your Student ID is filled in at the top of this page.
- Use the Print / Save as PDF button above to export this completed sheet.
- Submit both files as instructed by your instructor.



FACULTY OF COMPUTING